>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 13

Value-Returning and void Functions · Scope

return · bool predicates · returning arrays and tuples · pure functions · recursion · block scope, lifetime, local functions

```
> dotnet run
call Factorial(4)
  call Factorial(3)
    call Factorial(2)
      call Factorial(1)
      return 1
    return 2
  return 6
return 24
```

Asst. Prof. Hasan Oğuz, PhD

Istanbul Okan University · Vocational School

2026–2027 Fall Semester

>_ LEARNING OBJECTIVES

By the end of this week you can …

return and void

return ends a method and hands back a value; void methods perform actions.

bool predicates

IsEven, IsPrime, IsValidTCKimlik: methods that answer a yes/no question.

Several results

Return arrays; return two values with out parameters or a tuple.

Compose, stay pure

Max(Max(a, b), c); pure functions vs side effects; PASS/FAIL tests.

Recursion

Base case + recursive step; trace the call stack of Factorial and Fibonacci.

Scope and lifetime

Block and method scope, CS0103 / CS0136, local vs static local functions.

A method: a named, reusable block of code

13_01_ReturnBasics · a value-returning method

```
static int Square(int x)
{
    return x * x;
}
```

calls

```
int sq = Square(7);          // argument 7
Console.WriteLine(sq);        // 49
PrintTitle("Early return");   // void: no value
```



static int
return type: the call produces an int



Square
name: PascalCase, says what it computes



(int x)
parameter: a local variable, gets a copy



return x * x;
hands the value back and ends the method



PART 1

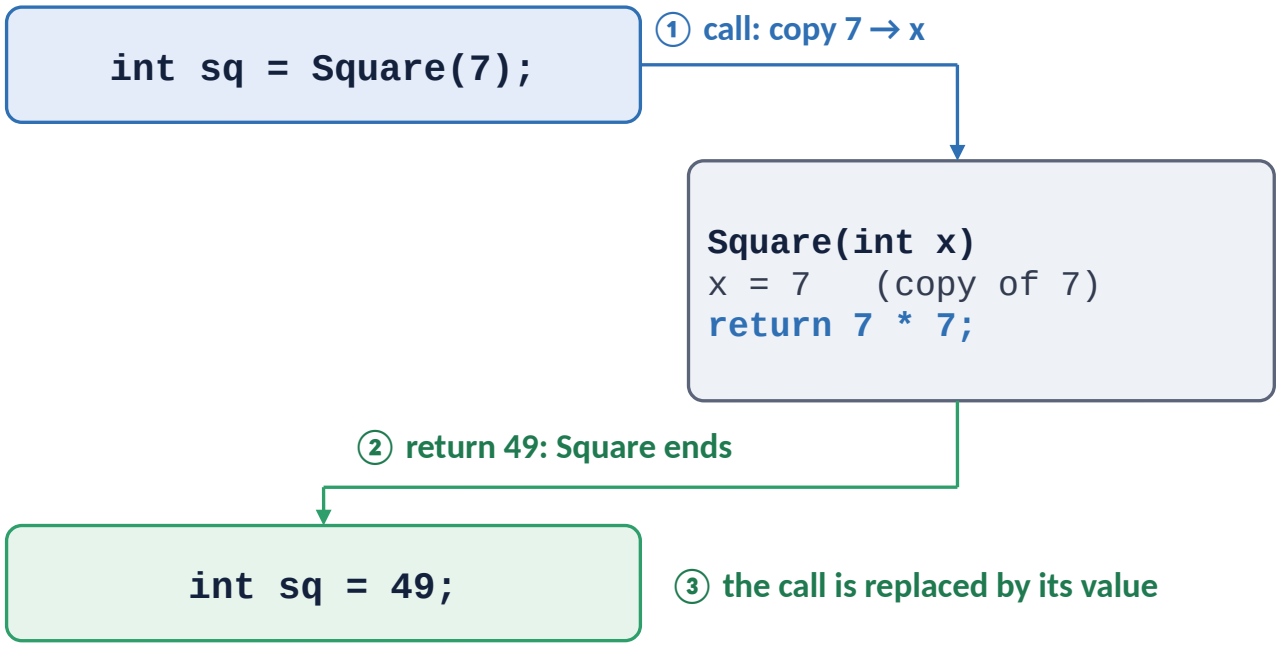
return and void

What a method gives back · early return · every path must return · bool predicates

return hands a value back — and leaves the method at once

Sign: three returns, one runs

```
static int Sign(int n)
{
    if (n > 0)
    {
        return 1;
    }
    if (n < 0)
    {
        return -1;
    }
    return 0;
}
```



Console

```
Square(7) = 49
Square(3) + Square(4) = 25
Sign(-8) = -1, Sign(0) = 0, Sign(5) = 1
```

An action or a computation?

void: an action (no value)

```
static void PrintTitle(string title)
{
    string line = new string('=', title.Length + 4);
    Console.WriteLine(line);
    Console.WriteLine($"  {title}");
    Console.WriteLine(line);
}
```

	void	value-returning
Purpose	action: print, change	computation: a result
Header	`void Show(…)`	`int Square(…)`
Call	a statement on its own	inside an expression
return	`return;`	`return value;`
return is …	optional	required on every path
Examples	PrintTitle, PrintArray	Square, Average, IsPrime



Prefer return over printing inside. Average(a) can be printed, compared with 60, stored or tested; PrintAverage(a) can only print.

dotnet build · using a void method as a value

```
int x = PrintTitle("A");
error CS0029: Cannot implicitly convert type
'void' to 'int'
```

Guard clauses: handle the bad case first, then return

guard clause + switch

```
static string LetterGrade(int score)
{
    if (score < 0 || score > 100)
    {
        return "invalid";
    }
    return score switch
    {
        >= 90 => "AA",
        >= 85 => "BA",
        >= 80 => "BB",
        >= 75 => "CB",
        >= 70 => "CC",
        >= 65 => "DC",
        >= 60 => "DD",
        _ => "FF"
    };
}
```

void: return; leaves early

```
static void PrintQuotient(int a, int b)
{
    if (b == 0)
    {
        Console.WriteLine($"{a} / {b}: cannot divide by zero");
        return;
    }
    Console.WriteLine($"{a} / {b} = {a / b} remainder {a % b}");
}
```

grades

```
 95 -> AA
 87 -> BA
 72 -> CC
105 -> invalid
```

PrintQuotient

```
17 / 5 = 3 remainder 2
17 / 0:
cannot divide by zero
```

The compiler checks every way out of the method

missing the last return

```
static int Sign(int n)
{
    if (n > 0)
    {
        return 1;
    }
    if (n < 0)
    {
        return -1;
    }
}   // n == 0: no return!
```

```
dotnet build
error CS0161: 'Sign(int)': not all code paths return a value
```

Code	Situation	Fix
CS0161	a path ends without return	add the last return
CS0127	`void method: return 0;`	return; or make it int
CS0126	`int method: return;`	return a value
CS0029	a void call used as a value	call it as a statement
CS0266	`int method: return n / 2.0;`	cast, or return double



Rule: a value-returning method must end with return on EVERY path — even the "impossible" one.

Methods that answer a yes/no question

return the condition itself

```
static bool IsEven(int n)
{
    return n % 2 == 0;
}
```

works, but unnecessary

```
if (n % 2 == 0)       // long-winded
{
    return true;
}
return false;
```

Console

```
IsEven(14) = True, IsEven(-7) = False
-29 % 10 = -9, but Mod(-29, 10) = 1
```

leave as soon as the answer is known

```
static bool IsAllDigits(string s)
{
    if (s.Length == 0)
    {
        return false;
    }
    foreach (char c in s)
    {
        if (c < '0' || c > '9')
        {
            return false;
        }
    }
    return true;
}
```

IsValidTCKimlik: the check digits, finally reusable

Pseudocode (Week 1)

```
IF length ≠ 11 OR not all digits
   OR d1 = 0 THEN
  RETURN false
END IF
odd ← d1+d3+d5+d7+d9
even ← d2+d4+d6+d8
d10 ← (7·odd − even) mod 10
d11 ← (odd + even + d10) mod 10
RETURN digit 10 = d10
   AND digit 11 = d11
```

13_02_TCKimlik

```
static bool IsValidTCKimlik(string id)
{
    if (id.Length != 11 || !IsAllDigits(id) || id[0] == '0')
    {
        return false;
    }
    int[] d = new int[11];
    for (int i = 0; i < 11; i++)
    {
        d[i] = id[i] - '0';
    }
    int oddSum = d[0] + d[2] + d[4] + d[6] + d[8];
    int evenSum = d[1] + d[3] + d[5] + d[7];
    int d10 = Mod(7 * oddSum - evenSum, 10);
    int d11 = (oddSum + evenSum + d10) % 10;
    return d[9] == d10 && d[10] == d11;
}
```

Seven synthetic test numbers — and the negative-mod trap

```
Console
12345678950  -> True
19090909018  -> True
12345678951  -> False
12345678590  -> False
02345678950  -> False
1234567895   -> False
1234567895A  -> False
Number to check: 12345678950
The check digits are correct.
```

19090909018	value
odd = 1+0+0+0+0	1
even = 9+9+9+9	36
7·odd − even	−29
-29 % 10	−9 ✗
Mod(-29, 10)	1 ✓ = digit 10
(1 + 36 + 1) % 10	8 ✓ = digit 11



One reason per test: valid · negative 7·odd − even · wrong d11 · swapped digits · leading 0 · 10 digits · a letter



Synthetic numbers only. Never use or share a real person's T.C. Kimlik No in code, tests or screenshots.



PART 2

More than one value · composition · purity

Returning arrays · out parameters and tuples · calls inside calls · side effects

A method can create an array and hand it back

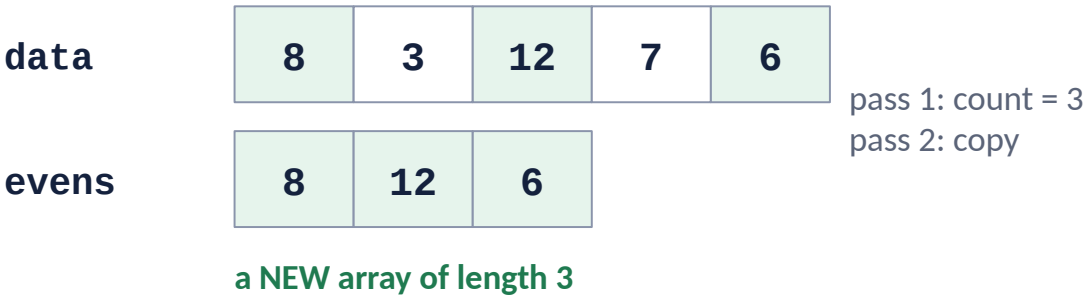
creates, fills, returns

```
static int[] ReadArray(int n)
{
    int[] result = new int[n];
    for (int i = 0; i < n; i++)
    {
        Console.Write($"  value {i + 1}: ");
        result[i] = int.Parse(Console.ReadLine());
    }
    return result;
}
```

```
int[] data = ReadArray(n);
int[] evens = EvensOnly(data);
```

Console

```
How many numbers? 5
  value 1: 8
  value 2: 3
  value 3: 12
  value 4: 7
  value 5: 6
data    : [8, 3, 12, 7, 6]  (Length 5)
reversed: [6, 7, 12, 3, 8]  (Length 5)
evens   : [8, 12, 6]  (Length 3)
data    : [8, 3, 12, 7, 6]  (Length 5)
```



Two results: out parameters or a tuple

out parameters

```
static void MinMaxOut(int[] a, out int min, out int max)
{
    min = a[0];
    max = a[0];
    // ... loop updates min and max
}
```

```
MinMaxOut(temps, out int low, out int high);
```



out: must be assigned inside — like int.TryParse.

a named tuple

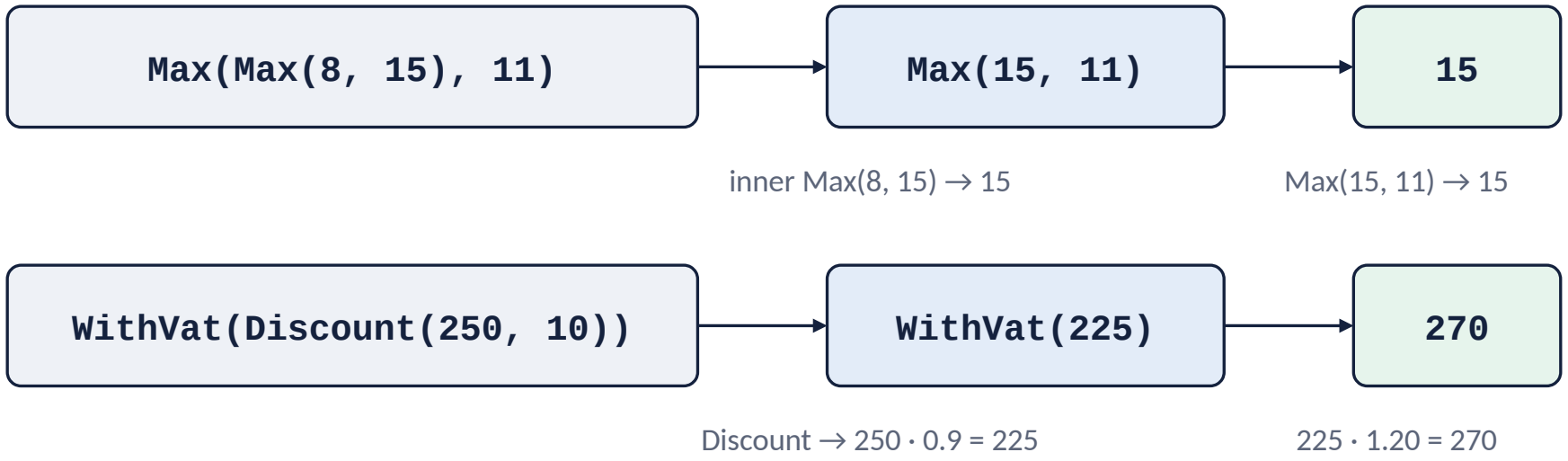
```
static (int min, int max) MinMax(int[] a)
{
    int min = a[0];
    int max = a[0];
    // ... loop updates min and max
    return (min, max);
}
```

```
(int min, int max) = MinMax(temps);
```



tuple: one value with named parts.

Calls inside calls: evaluated from the inside out



Inside out: the inner call runs first; its value is the argument.

13_05_PureFunctions

```
Max(Max(8, 15), 11) = 15
WithVat(Discount(250, 10)) = 270.00 TL
```

13_04_MultipleResults

```
temps: 14 9 21 17 6 12
out:   min = 6, max = 21
tuple: min = 6, max = 21, range = 15
whole: result.min = -5, result.max = 8
47 = 6 * 7 + 5
```

Same input, same output — and nothing else changes

pure vs side effect

```
static double Average(int[] a)
{
    int sum = 0;
    foreach (int x in a)
    {
        sum += x;
    }
    return (double)sum / a.Length;
}

static double CurveAndAverage(int[] a)
{
    for (int i = 0; i < a.Length; i++)
    {
        a[i] += 5;
    }
    return Average(a);
}
```

Console

```
Average: 73.67, again: 73.67
Passed? True
Average after Curve: 78.67
Average after Curve: 83.67
grades is now: 82 68 101
```

Pure
result depends only on the arguments; changes nothing

Side effect
prints, changes an argument array or a global variable



PART 3

Recursion

A method that calls itself · base case · the call stack · Fibonacci naive vs loop

Solve a problem with a smaller copy of itself

$$n! = n \cdot (n - 1)! \qquad 0! = 1! = 1$$

13_06_RecursionBasics

```
// n! = n * (n-1)!, with 0! = 1! = 1
static long Factorial(int n)
{
    if (n <= 1)
    {
        return 1;
    }
    return n * Factorial(n - 1);
}
```

Base case
n <= 1: answer directly, no further call.

Recursive step
call itself with a smaller n: n − 1.

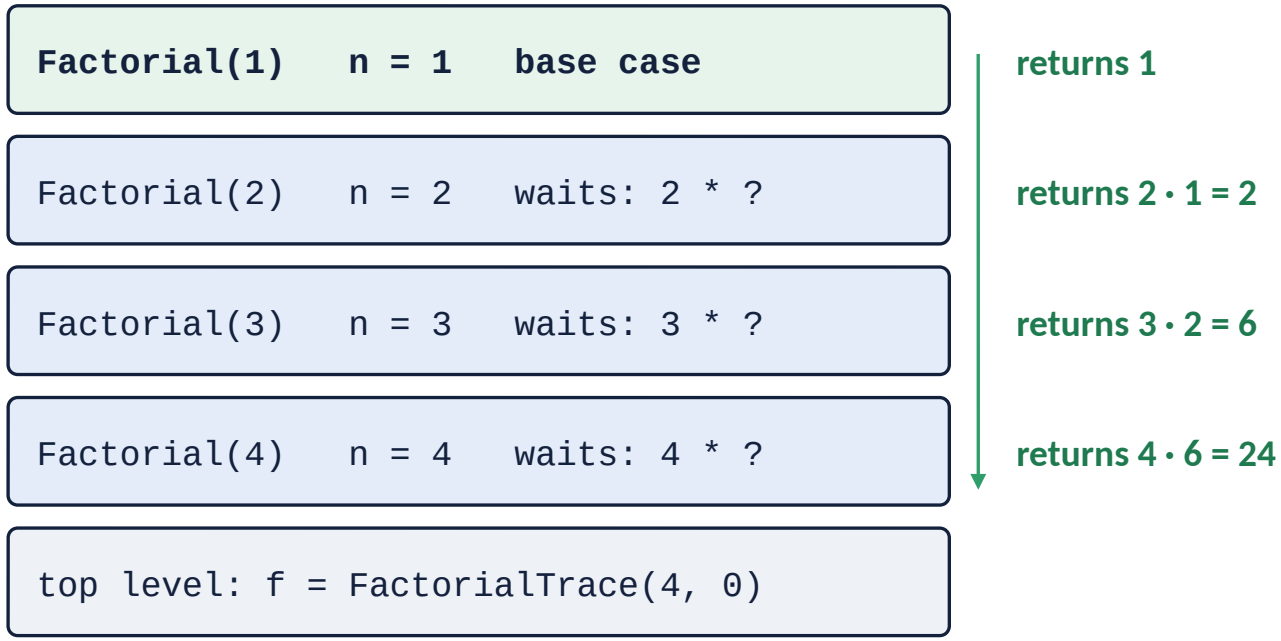
Console

```
Factorial(5) = 120
SumOfDigits(2026) = 10
Gcd(84, 36) = 12
```

Trace of Factorial(4): frames pile up, then unwind

Winding: each call pushes a frame. **Unwinding:** each return pops one.

top of the stack



Console

```
Call stack of FactorialTrace(4):
call Factorial(4)
  call Factorial(3)
    call Factorial(2)
      call Factorial(1)
      return 1
    return 2
  return 6
return 24
Result: 24
```

Sum of digits and Euclid's GCD, recursively

SumOfDigits

```
static int SumOfDigits(int n)
{
    if (n < 10)
    {
        return n;
    }
    return n % 10 + SumOfDigits(n / 10);
}
```

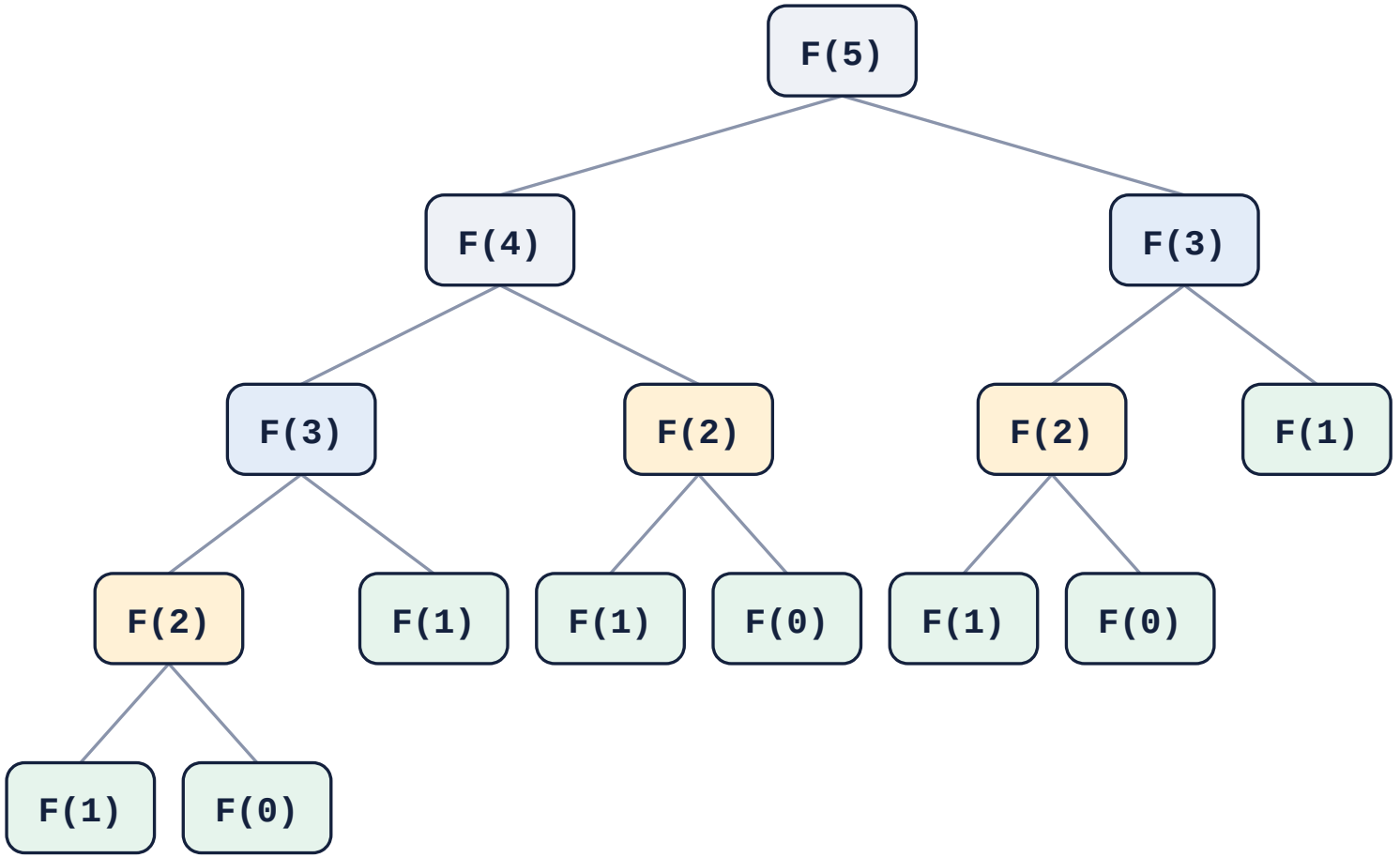
call	computes
SumOfDigits(2026)	6 + SumOfDigits(202)
SumOfDigits(202)	2 + SumOfDigits(20)
SumOfDigits(20)	0 + SumOfDigits(2)
SumOfDigits(2)	2 (base case)

Gcd (Euclid, Week 1)

```
static int Gcd(int a, int b)
{
    if (b == 0)
    {
        return a;
    }
    return Gcd(b, a % b);
}
```

call	a % b	result
Gcd(84, 36)	12	Gcd(36, 12)
Gcd(36, 12)	0	Gcd(12, 0)
Gcd(12, 0)	—	12 (base case)

The call tree of FibRec(5): the same work again and again



call	times
F(5)	1
F(4)	1
F(3)	2
F(2)	3
F(1)	5
F(0)	3
total	15

Repeated work. calls(n) = 2·Fib(n+1) − 1

Same numbers — 29 860 703 calls vs 34 loop passes

Console

```
First 10 terms: 0 1 1 2 3 5 8 13 21 34
  n        Fib(n)   recursive calls   loop passes
  5             5               15             4
 10            55              177             9
 20          6765            21891            19
 30        832040          2692537            29
 35       9227465         29860703            34
```

naive recursion: two calls per call

```
static long FibRec(int n, ref long calls)
{
    calls++;
    if (n < 2)
    {
        return n;
    }
    return FibRec(n - 1, ref calls) + FibRec(n - 2, ref calls);
}
```

loop: n − 1 passes

```
static long FibLoop(int n)
{
    long previous = 0;
    long current = 1;
    if (n == 0)
    {
        return 0;
    }
    for (int i = 2; i <= n; i++)
    {
        long next = previous + current;
        previous = current;
        current = next;
    }
    return current;
}
```

No base case: the stack runs out

builds with 0 warnings!

```
// infinite recursion: no base case
Console.WriteLine(Factorial(5));

static long Factorial(int n)
{
    return n * Factorial(n - 1);
}
```

```
dotnet run · process aborted (exit code 134)
Stack overflow.
Repeated 174131 times:
--------------------------------
   at Program.<<Main>$>g__Factorial|0_0(Int32)
```



Base case
Every recursive method needs a case that does not call itself.



Progress
Each call must move towards the base case (n − 1, n / 10, a % b).



Depth
Thousands of nested calls are fine, millions are not: use a loop.

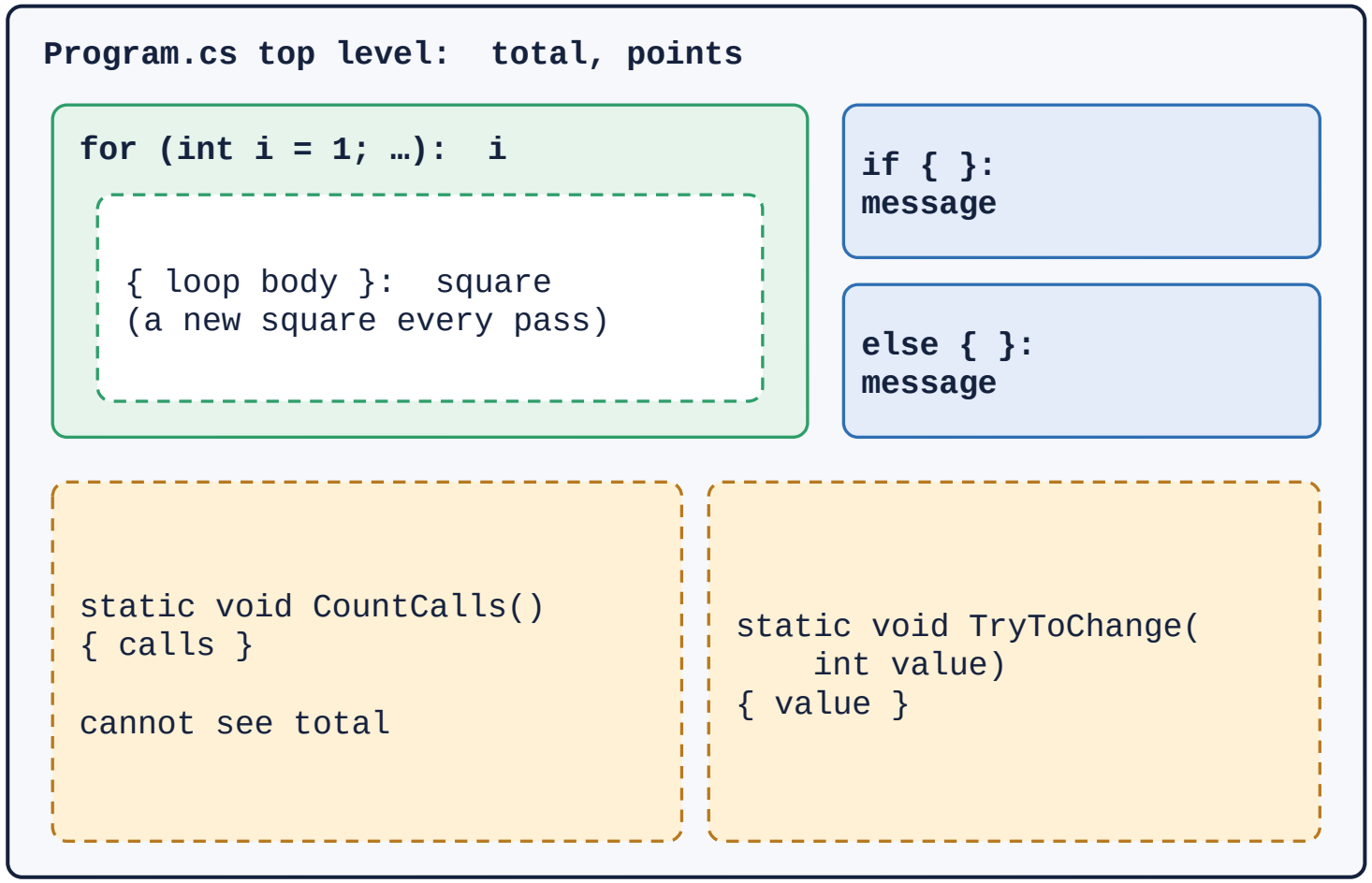


PART 4

Scope and lifetime

Where a name is visible · how long a variable lives · local functions · tests

Every { } is a box: you can look out, not in



Look out: allowed
An inner block sees the names of every enclosing block.

Look in: CS0103
Outside a block its names do not exist.

Siblings: independent
if and else may both declare message.

Static method: own box
Only parameters, locals and constants.

CS0103, CS0136, CS0128: one name, one meaning

13_08_BlockScope (comments shortened)

```
int total = 0;
for (int i = 1; i <= 3; i++)
{
    int square = i * i;
    total += square;
}
// Console.WriteLine(square);
// Console.WriteLine(i);
Console.WriteLine($"total = {total}");

for (int k = 0; k < 2; k++)
{
    // int total = k;      // CS0136
    Console.WriteLine($"k = {k}, total = {total}");
}
// int total = 5;          // CS0128
```

dotnet build

```
error CS0103: The name 'square' does not exist in the current context
error CS0103: The name 'i' does not exist in the current context
```

dotnet build

```
error CS0136: A local or parameter named 'total' cannot be declared in this scope because that name is used in an enclosing local scope to define a local or parameter
```

dotnet build

```
error CS0128: A local variable or function named 'total' is already defined in this scope
```

Locals are born at the call and die at the closing brace

a new calls at every call

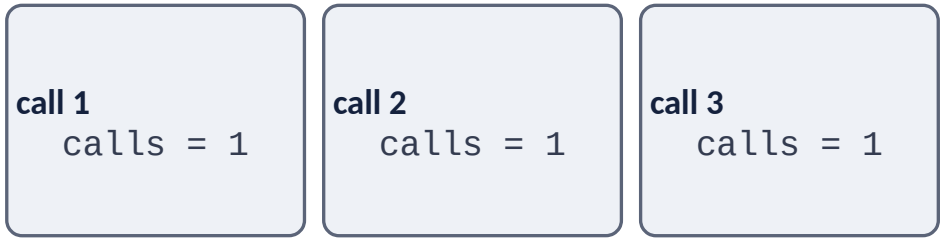
```
// Prints how often it has been called... or does it?
static void CountCalls()
{
    int calls = 0;
    calls++;
    Console.WriteLine($"CountCalls: calls = {calls}");
}
```

value: a copy of the argument

```
// Changes only its own copy of the argument
static void TryToChange(int value)
{
    value = 100;
    Console.WriteLine($"inside TryToChange: value = {value}");
}
```

Console

```
CountCalls: calls = 1
CountCalls: calls = 1
CountCalls: calls = 1
inside TryToChange: value = 100
after TryToChange: points = 40
```



Each box: calls is created with 0, becomes 1, and is destroyed when the method returns.

Capturing a variable vs a static local function

13_09_LocalFunctions

```
int AddBonus(int grade)
{
    return grade + bonus;
}

static int AddPoints(int grade, int points)
{
    // return grade + bonus;  // CS8421
    return grade + points;
}
```

```
dotnet build · static uses bonus / call before int bonus = 5;

error CS8421: A static local function cannot contain a
reference to 'bonus'.
error CS0165: Use of unassigned local variable 'bonus'
```

Console

```
AddBonus(70) = 75
AddBonus(70) = 80
   (same call, new result!)
AddPoints(70, 5) = 75
visits = 2
Square(9) = 81, top-level n is still 4
```



non-static: captures
reads/changes top-level variables (hidden input)



static: self-contained
only parameters + constants: preferred

Share constants freely — avoid shared variables

a constant: fine everywhere

```
const double VatRate = 0.20;

// Returns the price with VAT added
static double WithVat(double price)
{
    return price * (1 + VatRate);
}
```

a shared variable: avoid

```
int visits = 0;
Visit();
Visit();

void Visit()
{
    visits++;
}
```

	const	top-level variable	parameter + return
changes?	never	anywhere, any time	only via return
visible in header?	no (but fixed)	no — hidden	yes
static method?	allowed	CS8421	allowed
easy to test?	yes	no	yes



Course rule: local functions are static. Pass inputs as parameters, give results back with return (or a tuple).

Known input, expected output: PASS or FAIL

the test helper

```
static bool Check(string description, bool ok)
{
    string verdict = ok ? "PASS" : "FAIL";
    Console.WriteLine($"{verdict}  {description}");
    return ok;
}
```

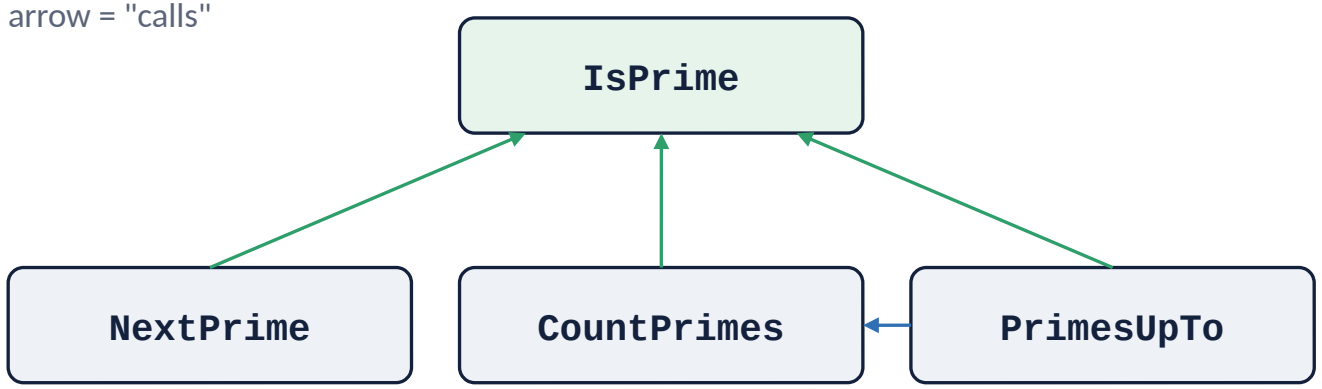
Console

```
--- Self-tests ---
PASS  Average {50, 70, 90} = 70
PASS  LetterGrade(90) = AA
PASS  LetterGrade(89.5) = BA
PASS  LetterGrade(59.9) = FF
PASS  GradePoint("CB") = 2.5
PASS  IsPassing("DD")
PASS  !IsPassing("FF")
7 of 7 tests passed
--- Report ---
Ayse      85.00  BA  3.5  passed
Mehmet    55.00  FF  0.0  failed
Zeynep    91.33  AA  4.0  passed
Can       50.00  FF  0.0  failed
```

```
    Check("LetterGrade(90) = AA", LetterGrade(90) == "AA"),
    Check("LetterGrade(89.5) = BA", LetterGrade(89.5) == "BA"),
    Check("LetterGrade(59.9) = FF", LetterGrade(59.9) == "FF"),
```

Utility methods: write once, call everywhere

arrow = "calls"



Console

```
Upper limit n: 50
Primes up to 50: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
Count: 15, next prime after 50: 53
--- Self-tests ---
PASS  IsPrime 1, 2, 97, 91
PASS  NextPrime(13) = 17, NextPrime(1) = 2
PASS  CountPrimes(100) = 25
```

Console

```
Enter a word or sentence: Kayak
Vowels: 2
Reversed: kayaK
Palindrome? True
36.6 C = 97.88 F, and back: 36.60 C
--- Self-tests ---
PASS  CountVowels("Programming") = 3
PASS  Reverse("abc") = "cba"
PASS  IsPalindrome("Ey Edip Adanada pide ye")
PASS  !IsPalindrome("Okan")
PASS  CelsiusToFahrenheit(100) = 212
PASS  round trip -40.5
All tests passed.
```

>_ COMMON MISTAKES

Ten return and scope bugs — and their fixes

Mistake	Symptom	Fix
a path without return	`CS0161`	end every path with return
printing inside a computing method	result cannot be reused or tested	return it; print in the caller
`Square(5);  (result ignored)`	no effect at all	`int sq = Square(5);`
`int y = PrintTitle("A");`	`CS0029 (void)`	a void call is a statement
`return n % 2 == 1;  // IsOdd`	false for −7	`n % 2 != 0`
using a variable after its block	`CS0103`	declare it before the block
same name in a nested block	`CS0136`	choose a different name
recursion without base case	Stack overflow (no warning)	base case + progress
method changes the array parameter	caller's data changed	work on a copy; say so
static method uses a top-level variable	`CS8421`	pass it as a parameter

>_ PREDICT THE OUTPUT

Quiz: what is printed?

Q1 · value or copy?

```
int x = 5;
Change(x);
Console.WriteLine(x);
Console.WriteLine(Twice(Twice(x)));

static void Change(int x)
{
    x = x * 10;
}

static int Twice(int v)
{
    return 2 * v;
}
```

Q2 · before and after

```
Show(3);

static void Show(int n)
{
    if (n == 0)
    {
        return;
    }
    Console.Write($"{n} ");
    Show(n - 1);
    Console.Write($"{n} ");
}
```

Q3 · early return

```
Console.WriteLine(Test(7));
Console.WriteLine(Test(-3));

static string Test(int n)
{
    if (n > 5)
    {
        return "big";
    }
    Console.WriteLine("checking...");
    if (n < 0)
    {
        return "negative";
    }
    return "small";
}
```

Answers (checked by running the code)

Q1

```
5
20
```

Q2

```
3 2 1 1 2 3
```

Q3

```
big
checking...
negative
```

Q1: x in Change is a copy; Twice(Twice(5)) = Twice(10) = 20.

frame	down	up
Show(3)	3	3
Show(2)	2	2
Show(1)	1	1
Show(0)	return	—

Q3: "checking..." is printed INSIDE Test(-3), before the caller prints the returned "negative".



Q2: code after the recursive call runs while the stack unwinds — in reverse order.

Quiz: which lines do not compile — and why?

Q4 · find three errors

```
 1  int sum = 0;
 2  for (int i = 1; i <= 4; i++)
 3  {
 4      int sum = i * i;
 5      sum += i;
 6  }
 7  if (sum > 5)
 8  {
 9      string label = "large";
10  }
11  Console.WriteLine($"{sum} {label} {i}");
```

Look for

names declared twice, names used outside their box

Draw the boxes

top level ⊃ for ⊃ body; top level ⊃ if-body

Bonus

Rewrite the program so it compiles and prints the sum of squares.

Three errors: one nested name, two names out of scope

```
dotnet build (real output, error lines only)
(4,9): error CS0136: A local or parameter named 'sum' cannot be declared in this scope because that name is used in an enclosing local scope to define a local or parameter
(11,28): error CS0103: The name 'label' does not exist in the current context
(11,36): error CS0103: The name 'i' does not exist in the current context
```

line	error	reason
4	CS0136	sum exists in the enclosing scope
11	CS0103	label only inside the if body
11	CS0103	i only inside the for

fixed → prints 30 large

```
int sum = 0;
for (int i = 1; i <= 4; i++)
{
    sum += i * i;
}
string label = "small";
if (sum > 5)
{
    label = "large";
}
Console.WriteLine($"{sum} {label}");
```

>_ IN-CLASS EXERCISES

Your turn (pairs, 25 minutes)

E1 · IsLeapYear + tests ★

Write bool IsLeapYear(int year) and four PASS/FAIL checks: 2024, 1900, 2000, 2026.

E2 · Recursive Power ★★

double Power(double b, int e) for e ≥ 0 with recursion. Trace Power(2, 5) on paper.

E3 · Three results ★★

Stats(int[] a) returns (int min, int max, double mean) as one tuple. Test with { 4, 8, 15, 16, 23, 42 }.

E4 · Scope detective ★

Predict the errors of the given snippet (in the notes) with their codes, then check with dotnet build.

>_ HOMEWORK

Before next week

H1 · Check-digit generator ★★

string CompleteTCKimlik(string first9) returns all 11 digits. Test: 123456789 → 12345678950; then IsValidTCKimlik(result).

H2 · ReadIntInRange ★★

int ReadIntInRange(string prompt, int min, int max) asks until TryParse succeeds and the value is in range.

H3 · Recursive binary search ★★★

int Find(int[] a, int target, int low, int high) returns the index or −1. Trace it for the Week 10 array.

H4 · Conversion table ★

Using CelsiusToFahrenheit, print a table for −10 … 40 °C in steps of 10 (F2 format).

Submit the four Program.cs files on the course group; include one console run for each.

>_ SUMMARY

Key Takeaways

01 return hands back one value and ends the method; every path must return (CS0161).

02 void methods act; value-returning methods compute. Prefer return over printing inside.

03 bool predicates read like questions: IsEven, IsPrime, IsValidTCKimlik.

04 Several results: return an array, use out parameters, or a tuple (int min, int max).

05 Recursion = base case + smaller step; each call has its own stack frame.

06 Scope: a name lives in its block; look out, never in (CS0103, CS0136). Prefer static local functions.

NEXT WEEK

Week 14: Files and semester review



MCMP113
Course materials